

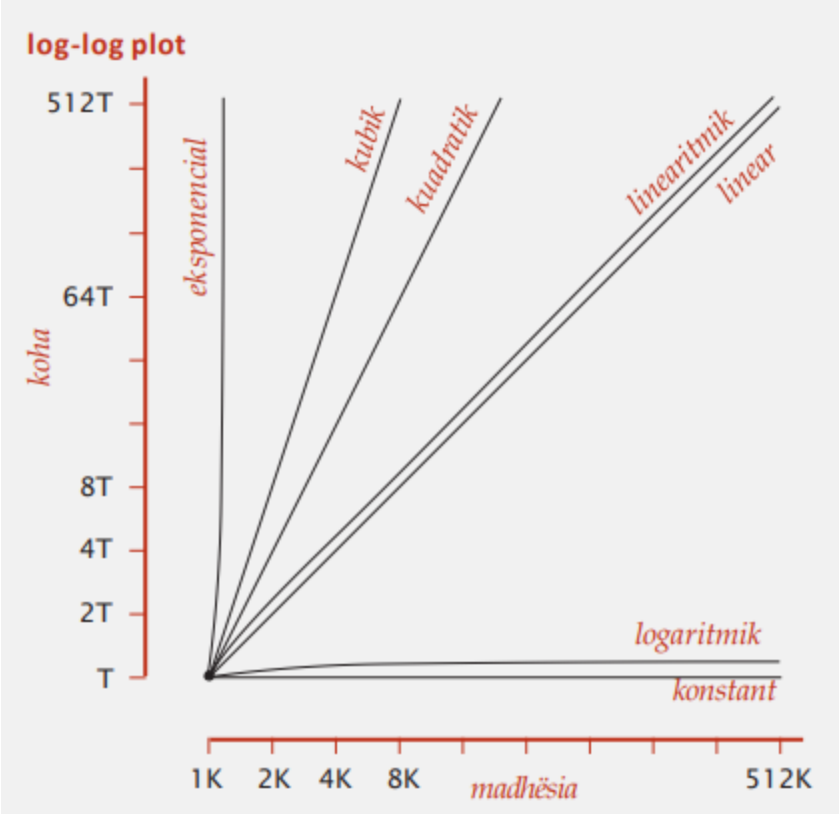
Java 2 - Detyrat e shtëpisë

Lënda: Hyrje në Algoritme

Hyrje:

- Big O Notation është një mjet që përdoret për të përshkruar kompleksitetin kohor të algoritmeve. Ai llogarit kohën e nevojshme për të ekzekutuar një algoritëm ndërsa hyrja rritet.
- Arsyet per analizim te kodit : Parashikimi i performancës, Krahasimi i algorithms, Garantimi i korrektësisë dhe Bazat teorike të algoritmeve.

Operacioni	Shembulli	Nanosekondat
Deklarimi i variablave	**int a**	c_1
Percaktimi i vleres	**a = b**	c_2
Krahasimi integer	**a < b**	c_3
Qasja në element të array	**a[i]**	c_4
Gjatesia e array	**a.length**	c_5
Alokimi i 1D array	**new int[N]**	$c_6 N$
Alokimi i 2D array	**new int[N][N]**	$c_7 N^2$



order of growth	name	typical code framework	description	example	$T(2N) / T(N)$
1	**constant**	`a = b + c;`	statement	add two numbers	1
$\log N$	**logarithmic**	`while (N > 1) { N = N / 2; ... }`	divide in half	binary search	~ 1
N	**linear**	`for (int i = 0; i < N; i++) { ... }`	loop	find the maximum	2
$N \log N$	**linearithmic**	[see mergesort lecture]	divide and conquer	mergesort	~ 2
N^2	**quadratic**	`for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) { ... }`	double loop	check all pairs	4
N^3	**cubic**	`for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) for (int k = 0; k < N; k++) { ... }`	triple loop	check all triples	8
2^N	**exponential**	[see combinatorial search lecture]	exhaustive search	check all subsets	$T(N)$

Detyra 1. Të gjendet kompleksiteti kohor për kodin e mëposhtëm dhe të vizatohet grafiku i funksionit algoritmik 0(n):

```
public static void calculateY(int nr){
        Int a = 0;
        Int b = 2;
        for (int i = 0; i < n; i++) {
                a++;
        }
}
```

Detyra 2. Të gjendet kompleksiteti kohor për kodin e mëposhtëm dhe të vizatohet grafiku i funksionit algoritmik 0(n):

```
public static void main(String args[]) {
        int num = 9;
        int i = 1;
        do {
                int value = num * i;
                System.out.println(num + " * " + i + " = " + value);
                i++;
        } while ( i <= 10 );
}
```

Detyra 3. Të gjendet kompleksiteti kohor për kodin e mëposhtëm dhe të vizatohet grafiku i funksionit algoritmik 0(n):

```
public static void main(String[] args){
        int count = 1;
        while (count < 11) {
                System.out.println("Count is:" + count);
                count++;
        }
}
```

Detyra 4. Të gjendet kompleksiteti kohor për kodin e mëposhtëm dhe të vizatohet grafiku i funksionit algoritmik 0(n):

```
public static int count(int[] a) {
        int n = a.length;
        int count = 0;
        for(int i = 1; i < n; i++){
                for(int j = 0; j < n; j++){
                        if (a[i] + a[j] == 0) {
                                count++;
                        }
                }
        }
        return count;
}
```

Vërejtje:

Detyrat e dhëna shkruhen ashtu si janë të dhëna në moodle, pastaj zgjidhja bëhet në fleta A4 **(me dorëshkrim).**

Detyrat e shtëpisë të dorëzohen javën e ardhshme tek asistentët përkatës fizikisht, në orën e ushtrimeve. Ju dëshirojmë shëndet dhe suksese!

#teamAlgo